Prajnaana Technologies
... embedded system solutions

PPG Signal Analysis Monitor

Technical Document

#40, 34th Cross, 11th Main, Jaya Nagar 4th T Block, Bangalore - 560041, Karnataka, India
Tel: +91-9448076994 Fax: +91-80-41210147 http://www.prajnaanatech.com

Table of Contents

PPG Signal Viewer - Technical Description

Version 1.10

Scope The document describes the Python implementation of the front end of the PPG signal processing done using the PPG Analysis C engine released by Prajnaana Technologies.

Audience Developers and reviewers who need to understand, modify or rebuild the system.

This document comes in two halves. The first explains how the system behaves when it runs. The second explains how it is built and how to extend it.

Part	Sections	Answers
I - Technical reference	1-8	What does each part do at runtime?
II - Construction	9-10	How is it built?

Licence and components

The tracker is released under the Apache Licence 2.0. The complete license text is provided in the LICENSE file located at the project root. Attribution details and third-party notices are provided in the NOTICE and CREDITS.md files.

The front-end needs PySide6 and pyqtgraph. Neither is redistributed here; section 9 explains how to install them.

Part I - Technical reference

1. Architecture

A note on the word "engine." Throughout this document, **the engine** means the compiled C program ppg_analysis: the part that reads a recording, filters it, finds the beats and computes the rates. The analysis of the PPG signal is contained entirely in the C implementation; visualisation is done in Python. The Python framework executes the C engine and plots the output of the analysis.

The system is therefore two programs and a single pipe between them.

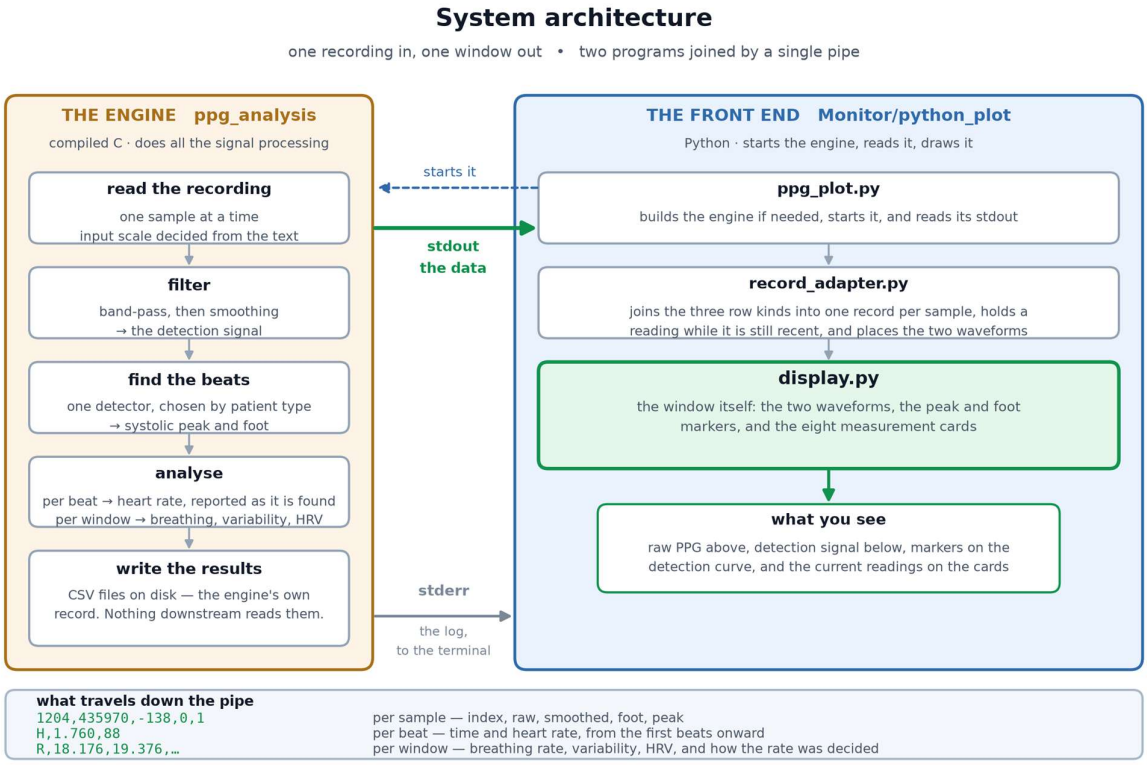

Figure 1 - The Python front end: Python spawns the engine and reads its stdout.

All data plotted is read from the interface pipe (stdout).

2. The interface protocol

The analysis engine produces two distinct types of output lines, with each line terminated by a line feed (LF). The type of each line is identified by its first character, allowing the reader to distinguish between the two-line types without requiring any additional information.

First character	Line type	Frequency
A digit	per sample	every sample of the PPG data
R	per window, indicating REPORT	once per analysis window, about every 8.2 s

The per-sample line is numeric comma-separated fields. The other two carry comma-separated alphanumeric values. All three are described below.

```
1204,435970,-138,0,1                    per sample, at the sampling rate
R,475.584,21.413,...,90,661.520,...,26   per analysis window, about every 8.2 s
```

Per-sample rows

Five fields, in the order given below.

Position	Field	Meaning
1	index	the sample number, counting from zero
2	raw	the recording as it arrived, after input scaling
3	smoothed	the signal the engine detected the beats on
4	foot	1 if this sample is a systolic foot, otherwise 0
5	peak	1 if this sample is a systolic peak, otherwise 0

Per-window rows

A line beginning R, carrying the engine's summary of the analysis window that has just closed.

Why one arrives roughly every 8.2 seconds. The respiratory analysis does not run on the raw samples. Beat fiducials are interpolated onto a uniform grid at **15.625 Hz** - that is 125 Hz divided by 8, the decimation the design is built around - and the analysis window advances **128 grid points** between reports:

```
128 grid points / 15.625 Hz = 8.192 s
```

Measured over the supplied adult recording, 56 intervals give a mean of 8.17 s. The small variation is because a window closes once enough interpolated points have accumulated, and those arrive per beat rather than on a clock.

What the row carries

#	Value	Meaning	Displayed as
1	Window time	Seconds of signal analysed when the window closed	status line
2	Respiratory rate	Breaths per minute, or -1 if no rate was reported	Respiratory Rate
3	Method	How the engine fused its estimates, or DECLINED	RR Status
6	Breathing-interval SD	Milliseconds	RR Variability
9	Heart rate	Beats per minute	Heart Rate
10	Mean NN interval	Average time between beats, ms	HRV Mean NN
11	NN SDNN	Spread of those intervals, ms	HRV SDNN
12	NN RMSSD	Successive differences between them, ms	HRV RMSSD

13	NN pNN50	Share differing by more than 50 ms	HRV pNN50

3. What appears on each stream

The engine is executed with the -p option enabled, which establishes a simple and consistent division of responsibilities within the pipe:

Stream	Carries
stdout	Data rows only, in the format of section 2
stderr	The engine's own console log. The front end neither captures nor redirects it, and names no file for it; it goes straight to the terminal.

4. Code description and functionality

The project consists of three files, which are maintained and updated independently.

```
Monitor/python_plot
    ppg_plot.py       -  entry point: builds, spawns, reads, shuts down
    record_adapter.py -  interprets the engine's output
    display.py        -  the Qt window
```

4.1 ppg_plot.py

This file is the entry point for the front end. It spawns the child process that runs the signal-processing engine, and carries the data between that engine and the visualisation framework. Nothing in it interprets the signal itself. The individual functions are described below.

Function	Description
run_display(result_queue, adapter)	Opens the window. Before showing it, adds the card for HRV mean NN and removes two unused rows from the plot legend.
project_root()	Returns the path of the project directory, located from this file's own position on disk.
find_engine(explicit)	Returns the path of the engine executable, ppg_analysis.exe on Windows. The project's own copy is preferred over any other copy on PATH. If the --exe command-line option was given, a relative path there is converted to an absolute one, because the engine is run from a different working directory.
engine_is_stale(engine)	Returns True if the engine executable is missing, or older than any src/*.c, include/*.h or the Makefile - that is, if it needs rebuilding.
build_engine(engine)	Runs mingw32-make, make or gmake, adding CC=gcc only when cc is not present.
build_command(args, engine)	Assembles the command line required to execute the engine. The -nu option is included only when a corresponding value is specified by the user.

spawn_engine(command, workdir)	Popen with stdout piped, stdin DEVNULL, stderr inherited. Catches WinError 1260 by name.
read_plot_stream(...)	The reader loop: parse, dispatch on the first character, pace, post to the queue, report the exit.
main()	Arguments, build, output directory, threads, the Qt loop, and shutdown.

Two constants sit at the top: DEFAULT_RATE_HZ (125) and DEFAULT_OUTDIR (run). **These must be changed if the sampling rate or the output directory changes.**

4.2 record_adapter.py

This file interprets the output of the analysis engine. **Any change to the interface has to be handled here**, and nowhere else.

Name	**Description**
RR_TIME_S … RR_MIN_PROM, RR_COLUMNS	The position of each field in the per-window row.
HR_TIME_S, HR_BPM, HR_COLUMNS	The two fields of the per-beat row.
HOLD_S	How long a reading survives a window that reported no rate, before the card goes back to --.
SURROGATE_NAMES	AM, BW, FM - used when saying which surrogate fell short.
_measured(text)	Turns one cell into a number, or into None for empty, 0.000 and -1.
_Track	One curve's baseline and envelope, both as running estimates, used to place it on screen.
BASELINE_TAU_S, ENVELOPE_HOLD_S, STACK_ROOM, STACK_GAP	The four constants that place the two curves on screen.
RecordAdapter.__init__(rate_hz, stack)	Empty metrics, no status, window_count at zero, and a track for each curve.
RecordAdapter.update_from_rr_row(fields)	Latches one per-window row. Returns False for a short row rather than padding it.
RecordAdapter.update_from_hr_row(fields)	Latches the heart rate from one per-beat row, so the card fills before the first window closes.
_rr_state(...)	Turns the engine's method into the phrase shown on the RR Status card.
_why(...)	Says which surrogate fell short, from the rates, the prominences and the two thresholds.
_warming(...)	Reports warm-up progress from the current and final window lengths.

RecordAdapter.record(index, raw, sample, foot, peak)	Builds one display record from one sample row, with the latched metrics attached.
RecordAdapter.status	The window time and fusion method, for the status bar.

4.3 display.py

This file handles the visualisation. It is **not** a third-party library - it is part of this project and is edited as the display needs it. What it does depend on are two external packages, and those are the standard ones for the job:

Package	**Purpose**	**Location**
PySide6	The window, the cards and the Qt event loop	https://pypi.org/project/PySide6/
pyqtgraph	The waveform plot and its markers	https://pypi.org/project/pyqtgraph/

The table below describes the functions used for visualisation.

Name	**Purpose**
MetricCard	One card: its title, its value, and the small note some of them carry.
LivePpgDisplay	The window itself: waveform, markers, cards, status bar.
LivePpgDisplay._read_results()	Drains the queue and applies each record.
LivePpgDisplay._format(value, unit, note)	Renders None as -- and any number as itself, with an optional note beneath - used to show how many intervals an HRV figure rests on.
LivePpgDisplay._on_curve(indices, oldest)	Pairs each marker index with the curve's own height at that sample.
LivePpgDisplay._prune(indices, oldest)	Drops markers that have scrolled off the left edge.
LivePpgDisplay._refresh()	The 50 ms timer tick: read, prune, redraw.
show_display(result_queue)	Creates the application and runs the Qt loop.

Three of its behaviours are load-bearing for this design, and must not be broken:

Behaviour	**Significance**
_format renders None as --	The not-measured rule in record_adapter only works because of this.
The status card reports a decision, not a number	It is fed a phrase such as " Strong match " or "Searching", so it must not be given a percentage.
_on_curve takes the marker height from the drawn curve	A marker cannot float off the trace, whatever amplitude is reported.

A record without raw_value is skipped	So record() must always set it.

The file also carries a compare mode that shows two detectors side by side. This front end does not use it: no record ever sets compare_engines, so those code paths stay dormant.

5. Threads

Thread	Functionality	Description
main	show_display() - the Qt event loop	Qt requires the main thread.
reader	stdout -> parse -> result_queue	Must not block the event loop.

Both are daemons watching a single threading.Event, so closing the window tears the session down along a path that already exists. On exit the engine is asked to terminate and then killed if it does not comply.

6. The adapter

Per-sample fields

Stream field	Display key
index	sequence
raw	raw_value - the grey curve, placed on top
smoothed	filtered_value and smoothed_value - the green curve, placed underneath
foot == 1	foot_detected, foot_index
peak == 1	peak_detected, peak_index

The three estimates

The engine works out the breathing rate three independent ways from the same pulse signal. The card names them when one falls short.

Name	Full name	What it measures, per beat
AM	Amplitude modulation	The height of each pulse. Breathing changes how much blood each beat pushes through, so the pulses grow and shrink with the breath.
BW	Baseline wander	The foot amplitude - where each pulse starts from. Breathing shifts the whole trace up and down as chest pressure changes venous return.
FM	Frequency modulation	The interval between consecutive maximal-upslope points. Breathing speeds the heart up on inhalation and slows it on exhalation.

FM is the weakest of the three, with a larger error than AM and BW on nearly every recording, because the heart-rate variation it measures is weak in adults. Moderate match with FM excluded is therefore an ordinary result, not a sign of trouble: the agreement gate leaving it out when it is wrong is what keeps the reported rate correct.

The RR Status card

The engine determines the processing outcome for each window.

What the card shows	What it means
Warming up 16 of 66 s	The analysis window has not grown to full length yet. The numbers are seconds: how much signal it has, out of what it needs.
Warming up	Still no rate to report, and no progress figure available - usually the very first window.
Strong match	All three of the engine's independent breathing estimates agreed. The most confidence available.
Moderate match	Two of the three agreed and were used; the third was set aside. A normal, usable reading.
Strong match - warm-up	As above, but measured on a window that is still growing, so treat it as provisional.
Moderate match - warm-up	The same caveat, on a two-of-three agreement.
Searching - BW differs	The estimates did not agree closely enough to report a rate, and this names the one that fell outside the band - AM, BW or FM, or a pair such as AM/FM.
Searching - all three differ	No two estimates agreed with each other at all.
Searching - BW/FM has no peak	Fewer than two estimates had a clear enough spectral peak to enter the vote. Different from disagreeing: these never voted.
Searching	The estimates did not agree, and the reason could not be worked out from the window.
No clear peak	No credible breathing rhythm was found in that window at all.

7. The input scale

A recording is plain text with no declared scale, so the engine needs to know whether it is reading integer counts or decimals. The -nu option controls this. Wrong configuration of -nu can lead to unexpected or faulty behaviour: on a decimal recording the wrong setting truncates every sample to zero, and no beat is found at all.

The front end does not pass `-nu` unless it is configured by the user. By default the engine works the scale factor out for itself and reports what it chose on its log:

```
Input scale: -nu 1000000 (auto: decimal text, 8 fractional digits, max |v| = 1 over 1024 samples)
Input scale: -nu 1 (auto: integer text, max |v| = 3131 over 1024 samples)
```

Those are the two recordings that ship in Monitor/, one written each way. Neither needed the option on the command line. The computation of the scale factor is outside the purview of this document.

Part II - Construction

8. Building the engine

The front end automatically builds the engine whenever the engine binary is missing or older than any file in src/*.c, include/*.h, or the Makefile. During this process, it searches for mingw32-make, make, or gmake and uses the first available build utility. The CC=gcc option is added only when the Makefile's default cc compiler is not available on the system PATH, which is the typical configuration of a MinGW installation.

If required, the same operation can be performed manually using the following procedure:

```
mingw32-make CC=gcc          # the normal build
mingw32-make strict CC=gcc   # adds -Werror -Wshadow
mingw32-make plot CC=gcc REC=<recording>
```

The project is built using the following compiler options:

```
-std=c99 -Wall -Wextra -Wpedantic
```

The code passes the make strict target without warnings or errors.

One portability consideration is worth noting. The functions dup, dup2, fileno, and fdopen are POSIX functions rather than part of the ISO C standard. Therefore, _POSIX_C_SOURCE is defined at the beginning of ppg_main.c, before the inclusion of the first system header. This ensures that the required POSIX interfaces are available when compiling on supported platforms.

The make asan target requires the MinGW toolchain to provide libasan and libubsan. If these libraries are unavailable, the target will fail during linking. This limitation is related to the capabilities of the installed toolchain and does not indicate an issue with the project source code.

Installing the toolchain on Windows

To build the C engine, a C compiler and a make utility are required. On Windows, the simplest way to obtain both is by installing MinGW-w64. Three installation options are provided below; select the option that best suits the development environment.

Option 1: winget (recommended). Windows 10 and 11 ship with winget, so this is usually a single command in PowerShell:

```
winget install BrechtSanders.WinLibs.POSIX.UCRT
```

What that package is. MinGW-w64 is a port of the GNU compiler collection that produces native Windows programs, and WinLibs is a prebuilt distribution of it maintained by Brecht Sanders. The parts of the name describe the build: **POSIX** is the threading model, and **UCRT** is Microsoft's Universal C Runtime, the current Windows C library rather than the older MSVCRT.

Why this one. The engine is plain C99 and needs only a C compiler and make, both of which this package provides in a single install, on PATH, with no further setup. UCRT matters because the engine's plot stream is opened in binary mode through the C runtime, and UCRT is the runtime current Windows builds use. This is also the exact toolchain the project was developed and tested against, so it is the one whose behaviour is known. Any MinGW-w64 that supplies gcc and a make will build it; the two alternatives below are equally valid.

Option 2: a plain download. Fetch a build from the WinLibs project at winlibs.com, unzip it wherever the user like, and add its mingw64\bin directory to project PATH.

Checking it worked. Open a new terminal, so that it picks up the updated PATH, and run:

```
gcc --version
mingw32-make --version
```

Both should print a version. If the shell reports that the command is not recognised, the PATH entry has not taken effect: close every terminal and open a fresh one, and if it still fails, Verify that the bin directory is included in the system PATH environment variable.

A note on the make command. MinGW-w64 names its make `mingw32-make`, whereas MSYS2 and Linux call it `make`. That is why every example here says `mingw32-make`, and why the front end looks for all three names before giving up. If the installation provides `make`, use that instead; nothing else changes.

Why `CC=gcc` appears in the commands. The Makefile asks for a compiler called `cc`. That is the conventional name for the system's default C compiler on Unix-like systems, where it is normally a link to whichever compiler is installed; it is not a separate program and not a cross compiler. A MinGW installation on Windows does not usually provide that name, so `CC=gcc` points the build at the compiler that is actually there. On a machine that does have a `cc`, the argument can be dropped.

Installing the Python dependencies

The front-end needs Python 3.10 or newer, plus two packages:

```
pip install PySide6 pyqtgraph
```

PySide6 supplies the window and pyqtgraph draws the plot inside it. Neither of these tools is required to run the C engine independently.